MaltaShop: A Practical, Phased Move to Microservices

Unit 5: Service-Oriented Architecture and Microservices Fundamentals

Student: Majharul Islam

Student ID: 11718

Institution: Learnkey Institute, Malta

Lecturer: Believe Ayodele

Submission date: 22 October 2026

Report body word count: 4,055 (references and contents excluded)

Contents

1. Architectural Evolution and Synthesis 3

1.1 Why the current monolith does not meet the brief . . . 3

1.2 Options, organisational fit and recommendation . . . 3

1.3 Distributed-systems consequences . . . 3

2. Scalable System Design 4

2.1 Business capabilities and target architecture . . . 4

2.2 Patterns and interaction choices . . . 5

2.3 Twelve-factor and cloud-native fit . . . 6

3. Lifecycle and Delivery Management 6

3.1 CI/CD design and release policy . . . 6

3.2 Order service test pyramid . . . 7

3.3 Payment deployment options and DORA measures . . . 8

4. Operational Excellence and Monitoring 8

4.1 Kubernetes, peak handling and operational risk . . . 8

4.2 Logs, telemetry, SLO and incident diagnosis . . . 9

4.3 Service mesh decision . . . 10

5. Technical Leadership and Collaboration 11

5.1 Board briefing (plain language) . . . 11

5.2 Team shape, collaboration and operating complexity . . . 11

5.3 Architecture Decision Records . . . 12

ADR 1 — Use RabbitMQ for operational events in the first migration . . . 12

ADR 2 — Standardise telemetry on OpenTelemetry and a small shared stack . . . 12

6. Implementation and Prototyping 13

6.1 Place-order flow and consistency . . . 13

6.2 Containers and production path . . . 13

6.3 Evidence, limitations and next steps . . . 14

Appendix A — Repository and evidence checklist 15

References 16

1. Architectural Evolution and Synthesis

1.1 Why the current monolith does not meet the brief

MaltaShop's PHP application has one deployment unit and shared MySQL schema, so unrelated functions scale and release together. At eight times normal Black Friday demand, checkout queued while the application ran on two virtual machines. Replicating the whole monolith adds capacity but also duplicates less-used work and does not remove database contention. Ten-times traffic needs an independently scalable checkout path, adequate database capacity and load tests to find bottlenecks.

The March notification memory leak shows the availability risk: a background email failure took checkout down because both shared a process. A separately deployed Notification service with bounded memory and a durable queue can restart without taking down Order. The seven-week payment-provider change shows coupling; a Payment service with a stable contract and provider adapter could be changed without editing customer and order internals. Independent boundaries enable separate scaling and releases, but do not make reliability automatic (Newman, 2021).

1.2 Options, organisational fit and recommendation

Keeping the monolith has the lowest immediate cost and is reasonable while MaltaShop adds tests and automates releases, but it does not provide independent scaling or fault isolation. An ESB centralises routing and transformation, which can suit many legacy integrations; here, vendor licensing and specialist skills are a risk, and business logic in the ESB can create a new central bottleneck. Microservices give capabilities independent deployment units but add network, consistency, security and monitoring costs (Richardson, 2018).

Twelve developers and two operations staff can form a few product teams, but not operate dozens of services or self-managed Kubernetes. The three-week, hand-operated release means tests, deployment automation and ownership must come first. I recommend a phased strangler transition: automate releases, extract Notifications first, then Payment and Order, and route traffic through a gateway. Keep the service count small, use managed infrastructure and require each extraction to show measurable benefit.

1.3 Distributed-systems consequences

Across a network, a timeout does not reveal whether a service failed before or after completing work. A broker, database or replica may fail while the rest remains available (partial failure). Slow links make bounded retries, timeouts, idempotency and duplicate messages normal concerns. Separate service stores cannot share a MySQL transaction, so MaltaShop must accept brief inconsistency, such as an order appearing before its notification.

Checkout should mean "the order is durably accepted," not that downstream work has finished. Persist the order and outbox event together, then publish asynchronously. A saga

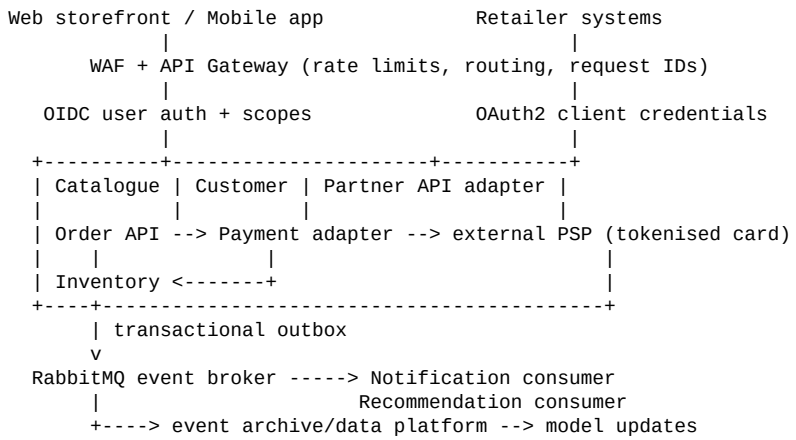
coordinates payment and stock with compensations; never retry a charge without an idempotency key. Traces, versioned contracts, replay-safe consumers and reconciliation add operating work, which must be funded (Newman, 2021; Richardson, 2018).

2. Scalable System Design

2.1 Business capabilities and target architecture

I would separate the platform by business capability and data ownership, not by technical layers. The table is a target state; the first releases can extract one boundary at a time.

Service	Responsibility	Data owned	Store
Catalogue	Product details, price and availability display	Product and listing records	Relational database; cache for read-heavy views
Customer	Account profile, preferences and consent	Personal data and consent history	Relational database with encryption and retention controls
Order	Order lifecycle and accepted line items	Orders and transactional outbox	Relational database
Payment	Provider adapters, authorisation and refund state	Provider references, payment state; no card PAN	Relational database; provider token/reference only
Inventory	Stock reservations and release	Per-SKU stock and reservations	Relational database
Partner API	Retailer identity, scopes, quotas and API translation	Partner configuration and audit records	Relational database
Notification	Email/SMS jobs and delivery status	Notification jobs and retry state	Relational database
Recommendation	Model inference and recommendation cache	Model version, features and disposable results	Object/model store plus cache; event consumer



```
Per-service relational stores; managed databases, not container filesystems
```

The gateway validates OIDC user tokens and scopes. Retailers receive separate OAuth2 client credentials, per-retailer scopes, rate limits and audit records. Services still authorise their own operations; gateway authentication does not make every internal caller trusted.

Browsing and outbox order events feed Recommendation asynchronously. Its queue and store are independent; lag or failure cannot block checkout. A timeout returns cached or generic suggestions, never a failed order.

2.2 Patterns and interaction choices

The gateway handles TLS, routing, authentication and retailer quotas but should not hold business logic. Database-per-service prevents cross-service joins and allows independent schema change. Publish-subscribe gives Notification and Recommendation separate consumers, so AI lag cannot block checkout. The transactional outbox commits the order and event intent together; a worker retries publication, and idempotent consumers tolerate redelivery. A saga coordinates payment authorisation and inventory reservation through explicit states and compensations rather than a distributed transaction (Richardson, 2018).

Interaction	Mode	Reason
App → Gateway → Catalogue	Synchronous REST	A product response is needed to render the page
App → Gateway → Order	Synchronous REST	Return a durable order acceptance and identifier
Order → Payment	Synchronous, bounded request for authorisation	Checkout needs an immediate accepted/declined decision; timeout and idempotency key required
Order → Inventory	Synchronous reservation request	Avoid accepting an order with known unavailable stock; compensate if later steps fail
Order → Notification	Asynchronous event	Email cannot delay checkout; retry independently
Order → Recommendation	Asynchronous event	Model and feature-processing delay must not affect checkout
Browsing → Recommendation	Asynchronous event	Near-real-time features without request-path coupling
Payment → Order status	Asynchronous event or callback	Provider settlement/refund updates can arrive later and must be reconciled

2.3 Twelve-factor and cloud-native fit

Immutable images receive configuration through environment variables; structured logs go to stdout; databases and RabbitMQ are separate backing services. API processes handle requests while workers publish and consume events. Health endpoints distinguish process health from readiness. This follows Twelve-Factor guidance on config, backing services, disposability, logs and processes (Wiggins, n.d.).

Cloud-native means automated provisioning, repeatable images, horizontal scaling and failure-aware services—not merely hosting in a cloud. Persistent data belongs on managed, backed-up services. Infrastructure as code, secrets, telemetry, zone redundancy and restore tests remain necessary. Prototype SQLite illustrates ownership only; production needs encrypted, highly available managed databases.

Migrate from the shared MySQL schema incrementally: document table ownership, remove cross-domain writes, then route each extracted capability through its API or events. Reconcile migrated records before retiring old tables. GDPR controls apply to events too: minimise personal fields and define deletion and retention responsibilities across services and backups (European Parliament and Council, 2016).

3. Lifecycle and Delivery Management

3.1 CI/CD design and release policy

For the Order service, developers work in short-lived branches and open pull requests to `main`. Branches should live hours or days. Peer review checks API contracts, privacy and operational requirements; incomplete features stay behind flags. Merging to main runs the production validation. `.github/workflows/ci.yml` installs dependencies, tests and builds both images. Production should also add static analysis, secret and dependency scans, SBOM publication and image signing.

```
Developer push / pull request
      |
  1. Format + static checks + secret/dependency scan
      | fail -> stop, report findings
  2. Unit tests (domain, outbox, idempotency)
      | fail -> stop
  3. Contract tests (OrderPlaced producer/consumer)
      | fail -> stop
  4. Integration tests (database + broker, disposable environment)
      | fail -> stop
  5. Build immutable image once; tag with commit SHA; scan + sign
      |
  6. Deploy same digest to staging -> smoke + migration compatibility gate
      | fail -> roll back staging; block promotion
  7. Production approval -> progressive rollout (canary)
      |
  8. SLO/error-budget checks + telemetry review -> complete or auto-rollback
```

The image built from a particular commit is promoted by digest through test, staging and production. Rebuilding separately for each environment can introduce different dependencies or source; promoting one artifact makes the tested object the deployed object.

Environment-specific configuration and secrets are injected at runtime. Database migrations follow an expand-and-contract sequence: first add a backward-compatible field, deploy readers/writers, backfill, then remove obsolete schema only after old versions have gone.

Pull requests require an owner review and passing contract test; changes to APIs or data include a migration note. Link the commit, image digest, migration and deployment event in the release record so responders can identify exactly what changed. Never put secrets in Git or image layers. Staging should use production-like configuration and masked test data.

I choose continuous delivery rather than fully automatic continuous deployment for the initial year. Every passing change is production-ready and can be released independently, but a human release owner approves production promotion while the team learns to operate the platform. Once rollback, alerting and on-call evidence are reliable, the board can approve automatic deployment for low-risk changes. This balances weekly releases with the current manual operational capability. Continuous delivery is not the old Friday outage compressed into more frequent releases; it is an always releasable system with small, reversible changes (Humble and Farley, 2010).

3.2 Order service test pyramid

Fast unit tests cover validation, state transitions, idempotency, outbox atomicity and compensation decisions without network dependencies. A contract test uses the versioned `OrderPlaced` schema (`event_id`, `event_type`, `order_id`, `occurred_at`, `correlation_id`, line items); Order and Notification both verify compatibility before a field change can break the consumer.

Disposable database and RabbitMQ integration tests verify transaction boundaries, publisher confirms, durable queues, acknowledgements and redelivery. A few end-to-end tests cover gateway authentication, eventual notification and duplicate submission. Staging load tests ramp to ten times baseline and measure latency, errors, queue depth and database saturation.

Test level	Example	Pipeline stage / frequency
Static / unit	Schema validation, state machine, idempotency	Every push
Contract	Order publisher ↔ Notification consumer event schema	Every pull request
Component / integration	Outbox, broker confirms, database, retries	Pull request; repeat before release
End-to-end	Authenticated place-order through visible notification job	Staging deployment
Load / resilience	Ten-times traffic; broker or Notification outage	Scheduled staging exercise and release readiness

The prototype unit tests cover local data and event logic. CI also starts the Compose stack and verifies an actual order request reaches Notification with the same correlation ID. Broader contract tests, failure-injection tests and ten-times-load tests remain release-readiness work.

Test repeated requests with the same idempotency key, broker outage, duplicate delivery, malformed events and consumer restart. A release gate should fail on a broken OrderPlaced contract or a migration incompatible with the prior version. Retain test results and artifact digests with the release for incident response and audit.

3.3 Payment deployment options and DORA measures

Rolling replaces pods gradually and saves capacity, but mixes versions and requires compatible schemas. Blue-green switches between two environments and rolls back quickly, at near-double capacity; it cannot undo incompatible writes. Canary exposes a small traffic share, checks errors and latency, then expands. It adds routing complexity but best limits risk for a payment adapter.

I would start Payment at 5% of eligible traffic, then move to 25%, 50% and 100% after agreed observation windows. Keep the stable version available. Roll back by directing all new requests to the stable version, disabling the canary feature flag, and checking in-flight provider requests using idempotency keys and provider references; never blindly retry a potentially completed charge. Reconcile authorization and capture state before replay. Database changes remain backward compatible until the canary is stable.

Track deployment frequency and lead time for changes (Forsgren, Humble and Kim, 2018). The baseline is one release per three weeks; target at least weekly releases without planned downtime. Adding the second payment provider took seven weeks, the only available lead-time example; it is a proxy, not a portfolio-wide DORA sample. Instrument ready-commit to production time and target a median under five working days within six months. Report definitions and sample size; do not invent missing failure-rate or recovery baselines.

4. Operational Excellence and Monitoring

4.1 Kubernetes, peak handling and operational risk

Stateless Order, Catalogue, Payment, Notification and Recommendation workloads run as Kubernetes Deployments. Deployments manage ReplicaSets, which restore desired replicas after pod or node loss. Separate resource budgets contain a Notification leak; topology spread and a checkout PodDisruptionBudget reduce disruption during maintenance.

Readiness probes control traffic eligibility; an optional recommendation cache should not make Order unready. Liveness restarts a stuck process, not one whose broker is briefly down; startup probes allow slow initialization. Measured CPU and memory requests guide

scheduling, while limits contain runaway processes. Tight CPU limits can throttle checkout. Horizontal Pod Autoscaling adds Order replicas using CPU or custom concurrency metrics, but reacts with delay; pre-size baseline replicas, nodes and database connections for a sale (Kubernetes, n.d.-a; Kubernetes, n.d.-b).

Managed relational databases and a managed or well-operated broker hold state outside Deployments; a StatefulSet does not replace backup. Recommendation scales independently and can be throttled. Ten-times capacity requires load tests, headroom, caching, rate limits, tuned connections and degradation tests—not autoscaling alone.

With no Kubernetes experience, start with a managed control plane and non-production cluster. Train operations and developers; rehearse node failure, restore and rollback. Migrate one low-risk service first and retain the VM fallback until evidence supports moving Order. Set spend alerts. If cost or training is prohibitive, use a simpler managed container platform first and revisit Kubernetes using measured demand.

For checkout, begin with at least three replicas spread across failure zones and set the HPA minimum above one to avoid cold-start delays during a sale. Choose the maximum only after tests confirm the database and provider can support it; extra API pods cannot fix a saturated database. Cap connection pools per replica, protect availability with a disruption budget, and load-shed nonessential work. Recommendation has separate limits and queues, preserving Order's reserved capacity.

4.2 Logs, telemetry, SLO and incident diagnosis

Services emit JSON to stdout with UTC `timestamp`, `level`, `service`, `environment`, `event`, `correlation_id`, `trace_id`, `request_id`, pseudonymous user/partner reference, `order_id`, `duration_ms`, `outcome` and `error_code`. Do not log card data, tokens, passwords, addresses or full payloads. Centralise redaction and retention. The gateway generates or accepts a correlation ID and propagates it in headers and event metadata; it is not an authentication credential.

Use OpenTelemetry instrumentation and collectors; Prometheus-compatible metrics with Grafana; and access-controlled central logs such as Loki. A managed service may reduce operations at scale, but avoid overlapping tools. Metrics show rate and latency, traces show the service path, and logs provide event detail, joined by trace and correlation IDs (Beyer et al., 2016).

The checkout SLO is 99.9% monthly availability: successful valid order-submission requests divided by all valid order-submission requests, excluding client-side validation errors but counting timeouts and server failures. On a 30-day month, 0.1% error budget is about 43 minutes. Two actionable alerts are:

Alert	Metric and threshold	Duration / severity	First runbook step
Checkout error burn	`checkout_requests {status=~"5.."} / checkout_requests > 1%`	5 minutes; SEV-1	Check gateway and Order dashboard; compare deploy markers and database saturation
Checkout latency	p95 `place_order` duration > 1 second	10 minutes; SEV-2	Inspect trace samples and payment-provider latency before scaling blindly

For the March leak, alert on Notification memory growth, restarts and queue age. The engineer follows the correlation ID; traces show the failure in asynchronous work, while Order readiness and checkout errors remain healthy. Pause or scale down the consumer, retain queued messages, deploy a fixed image and monitor backlog recovery. The review adds a regression test and explains why the leak escaped. Service isolation keeps checkout available while email is delayed.

Measure availability at the customer-facing order endpoint, not by counting healthy pods. A synthetic transaction checks the gateway path while request metrics calculate the monthly SLO. A burn-rate dashboard, deployment markers and queue-depth panel distinguish delayed email from a growing payment backlog. Every alert names its owning team so the incident does not default to Operations.

4.3 Service mesh decision

A service mesh places a proxy beside each workload for mutual TLS, telemetry, traffic shifting and policy-controlled retries, timeouts and circuit breaking. TLS supplies workload identity; bounded, jittered retries help transient failures but must never repeat a non-idempotent charge. Timeouts bound waits; circuit breakers stop calls to unhealthy dependencies.

Do not deploy a mesh initially. Sidecars add CPU, certificates, upgrades and debugging effort to a small, inexperienced team. First implement deadlines, idempotency, bounded retries, circuit breakers and telemetry in the gateway and clients. Reconsider a mesh only when service count and security needs justify it; it cannot fix unsafe policies or unclear ownership (Newman, 2021).

5. Technical Leadership and Collaboration

5.1 Board briefing (plain language)

MaltaShop's current platform is like one large shop where every department shares the same doors, storage and closing time. It has been inexpensive to understand, but a problem in email stopped checkout, a sale overwhelmed the shared system, and a payment-provider change took seven weeks. We should not replace everything at once or buy a large integration platform. Instead, we should improve automated releases first and separate the parts where the case shows real pain, beginning with notifications and payments.

This change costs engineering time, managed hosting, monitoring and training. It also creates new risks: messages can arrive late or twice, and more components need watching. We will control those risks with small steps, named owners, automatic tests, backups and a rollback route. We will use managed services rather than asking our two operations colleagues to build a Kubernetes platform alone.

Microservices are not automatically cheaper, and they would be overkill if we created many tiny services with no reason. Here, independent checkout scaling, safe weekly releases, and preventing email failures from stopping orders are specific business benefits. We will continue only when each extraction improves a measured outcome. The board will see monthly cost, checkout availability, deployment frequency and incidents, and can pause the transition if the benefits do not justify the spend.

5.2 Team shape, collaboration and operating complexity

I would organise the twelve developers into three stable, cross-functional groups of four: Commerce (Catalogue and Recommendation), Transaction (Order, Payment and Inventory), and Customer & Integration (Customer, Partner API and Notification). The boundaries are initial ownership areas, not an instruction to create eight isolated teams. Each team owns code, tests, dashboards, runbooks and service-level objectives for its capabilities. The two operations staff form a platform pair, maintaining reusable pipelines, deployment templates, access control, observability and cloud cost controls. They coach teams; they do not become the only people allowed to deploy.

Each group should have a named service owner and rotate another developer through operational work. The platform pair runs short workshops on logs, rollback and incident simulations, then checks that each squad can deploy and restore without intervention. A monthly architecture forum resolves cross-team API and data ownership questions, while service owners retain accountability. The board receives cost and SLO trends in plain language, not a count of containers.

Use protected `main`, short-lived branches, peer review and required CI. Shared event-schema changes need a consumer reviewer; OpenAPI and versioned event schemas document

contracts. Breaking changes require a compatibility window. A service catalogue records owner, data classification, dependencies, SLO, alerts and runbook; ADRs preserve decision context.

Begin with business-hours on-call and a trained service owner, escalating to the platform pair until squads complete drills. Fund a rota before promising 24/7 coverage. Review material incidents within five working days and assign corrective owners; responders must be able to pause a risky rollout.

Three complexities need controls: delayed or duplicate events require outbox publishing, deduplication and reconciliation; more services require owner-backed alerts and a shared service map; distributed privacy and access require central identity, secret rotation and audit retention, with each owner accountable for its data.

5.3 Architecture Decision Records

ADR 1 — Use RabbitMQ for operational events in the first migration

Context: Publish OrderPlaced to Notification and Recommendation without blocking checkout; the team has limited platform experience and budget. **Decision:** Use durable RabbitMQ queues, publisher confirms, consumer acknowledgements, an outbox and idempotent consumers. Archive analytics events separately. **Alternatives rejected:** A managed cloud queue (less operations, more provider coupling/cost); Kafka (more replay but unnecessary overhead now); synchronous HTTP fan-out (couples checkout to consumers). Criteria: delivery, familiarity, cost and near-real-time delivery. **Consequences:** Operate a highly available broker and monitor queue age and redelivery. Accepted debt is limited replay compared with a streaming log; revisit if volume or replay needs grow.

ADR 2 — Standardise telemetry on OpenTelemetry and a small shared stack

Context: Local files and absent alerts leave incidents to customer reports; distributed services need correlated, privacy-aware evidence. **Decision:** Instrument with OpenTelemetry and centralise metrics, traces and access-controlled structured logs. Start with Prometheus/Grafana/Loki-compatible tools or one managed alternative after cost and retention review. **Alternatives rejected:** Local files (not searchable); separate vendors for each signal (fragmented and costly); per-team logging (schema drift). Criteria: correlation, open formats, privacy, learning curve and cost. **Consequences:** Teams own instrumentation, dashboards and retention. OpenTelemetry limits vendor lock-in, but storage still costs money. Accepted debt is initial capacity tuning; portable schemas make backend replacement possible.

6. Implementation and Prototyping

6.1 Place-order flow and consistency

The two prototype services have separate SQLite files. `POST /orders` validates items and commits an order plus outbox row in one transaction. A publisher sends the event to a durable RabbitMQ queue with confirms; Notification persists a job before acknowledging. This is at-least-once: a crash after broker acceptance may cause redelivery, which the consumer's unique `event_id` deduplicates. `Idempotency-Key` prevents duplicate orders on client retry.

RabbitMQ's durable queue and persistent-message setting support recovery after a broker restart when durable storage is configured; publisher confirms indicate the broker accepted responsibility (RabbitMQ, n.d.). Notification acknowledges only after its database commit. This is at-least-once delivery, not exactly-once business effect, so the event ID and unique constraint matter. The prototype retries via its outbox poller; production should add jittered backoff, dead-letter handling, queue-age alerts and bounded shutdown.

The important producer logic is:

```
connection.execute("BEGIN IMMEDIATE")
connection.execute("INSERT INTO orders (...) VALUES (...)")
connection.execute(
    "INSERT INTO outbox(event_id, order_id, event_json, created_at) VALUES (?, ?, ?, ?)",
    (event_id, order_id, json.dumps(event), now),
)
connection.commit()
```

These rows commit together, so there cannot be a committed order with no durable record of the event to publish. The event includes the same `correlation_id` logged when the API accepts the order. The consumer performs an idempotent insert:

```
cursor = connection.execute(
    "INSERT OR IGNORE INTO notification_jobs "
    "(event_id, order_id, correlation_id, status, created_at) VALUES (?, ?, ?, 'QUEUED', ?)",
    (event["event_id"], event["order_id"], event["correlation_id"], now),
)
channel.basic_ack(delivery_tag=method.delivery_tag)  # only after commit succeeds
```

Production Order–Payment–Inventory needs a saga: Payment authorises with an idempotency key, Inventory reserves, then Order confirms and Payment captures. Stock failure voids authorisation; payment failure releases stock. A timeout requires status lookup, not blind retry. Compensation can also fail, so alert on `COMPENSATION_REQUIRED`. The prototype does not implement real payment, inventory, saga orchestration or production concurrency controls.

6.2 Containers and production path

Each service has a Dockerfile and pinned dependencies based on Python 3.12 slim, runs as non-root and exposes only its needed port. Named volumes hold demo SQLite files. The Order image has a health check and Compose waits for RabbitMQ. Credentials and database paths are configurable; the local default password is not a production secret.

`prototype/compose.yaml` starts RabbitMQ, Order and Notification with one command; README documents the request and logs. CI tests and builds images. Production adds a registry, image scanning/signing, secrets manager, backed-up managed databases, highly available broker and container platform. Infrastructure as code defines network boundaries, encryption, identity, scaling and alarms; promotion uses an immutable image digest and compatible migrations.

Production also needs a gateway/WAF for retailer quotas, an identity provider for OIDC/OAuth2, certificate rotation and central telemetry collection. A secrets manager injects broker and provider credentials at runtime. Test database restoration rather than treating a successful backup job as proof. These controls need operational owners; a Dockerfile does not provide them automatically.

The small Dockerfiles use single-stage builds; if compilation dependencies grow, copy only runtime artifacts into a clean final stage. Health checks do not prove dependencies are healthy. Separate readiness and liveness, and keep Order ready during broker failure because the outbox can buffer events. Production must replace SQLite, rotate secrets, tune pools and test recovery.

6.3 Evidence, limitations and next steps

The repository includes two services and stores, REST and asynchronous event paths, Dockerfiles, Compose, CI and tests for atomic outbox persistence, idempotency, event validation, deduplication and correlation IDs. Run tests with `python -m pytest prototype/order_service/test_app.py prototype/notification_service/test_consumer.py -q` after installing requirements and pytest.

Required evidence is a real screenshot of (1) the order request/response, (2) Notification logs with the same correlation ID, and (3) a passing GitHub Actions run. Docker is not installed here and no Actions run was executed, so I have not fabricated this evidence. Before submission, run Compose on a Docker host, capture the request and logs, push to the provided repository and attach a screenshot of the actual green workflow. Unit tests do not prove the broker path end to end.

The request image should show the command and accepted JSON response with correlation ID; the consumer image should show the matching ID and `notification_queued` record. The CI image should identify the repository, workflow run and green status. Add captions and timestamps in an evidence appendix, and do not label unit-test output as a broker-backed demonstration.

The prototype demonstrates asynchronous notification, correlation IDs and handling of duplicate delivery. It lacks real payment/inventory orchestration, authentication, rate limits,

schema governance, production databases, broker HA, dead-letter operations, load tests, secret management and telemetry export. Before launch, add broker-backed contract tests, production storage, failure-tested compensation, ten-times load testing, privacy review and rollback/restore drills. Do not claim 99.9% until measured.

Appendix A — Repository and evidence checklist

The report source and prototype are published at https://github.com/majharulislam783/soa-assignment. GitHub Actions runs unit tests and a Compose end-to-end check, publishes the actual order response and both service logs to the run summary, and uploads the same files as an artifact. The next run's summary can be captured as genuine screenshots of the passing workflow, the accepted order response and the matching correlation ID across Order and Notification.

References

Beyer, B., Jones, C., Petoff, J. and Murphy, N.R. (eds.) (2016) *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA: O'Reilly Media.

European Parliament and Council of the European Union (2016) 'Regulation (EU) 2016/679 (General Data Protection Regulation)', *Official Journal of the European Union*, L 119, pp. 1–88. Available at: https://eur-lex.europa.eu/eli/reg/2016/679/oj (Accessed: 20 October 2026).

Forsgren, N., Humble, J. and Kim, G. (2018) *Accelerate: The Science of Lean Software and DevOps*. Portland, OR: IT Revolution Press.

Humble, J. and Farley, D. (2010) *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston, MA: Addison-Wesley.

Kubernetes (n.d.-a) 'Horizontal Pod Autoscaling'. Available at: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/ (Accessed: 20 October 2026).

Kubernetes (n.d.-b) 'Liveness, Readiness, and Startup Probes'. Available at: https://kubernetes.io/docs/concepts/configuration/liveness-readiness-startup-probes/ (Accessed: 20 October 2026).

Newman, S. (2021) *Building Microservices: Designing Fine-Grained Systems*. 2nd edn. Sebastopol, CA: O'Reilly Media.

RabbitMQ (n.d.) 'Reliability Guide'. Available at: https://www.rabbitmq.com/docs/reliability (Accessed: 20 October 2026).

Richardson, C. (2018) *Microservices Patterns: With Examples in Java*. Shelter Island, NY: Manning Publications.

Wiggins, A. (n.d.) 'The Twelve-Factor App'. Available at: https://12factor.net/ (Accessed: 20 October 2026).